

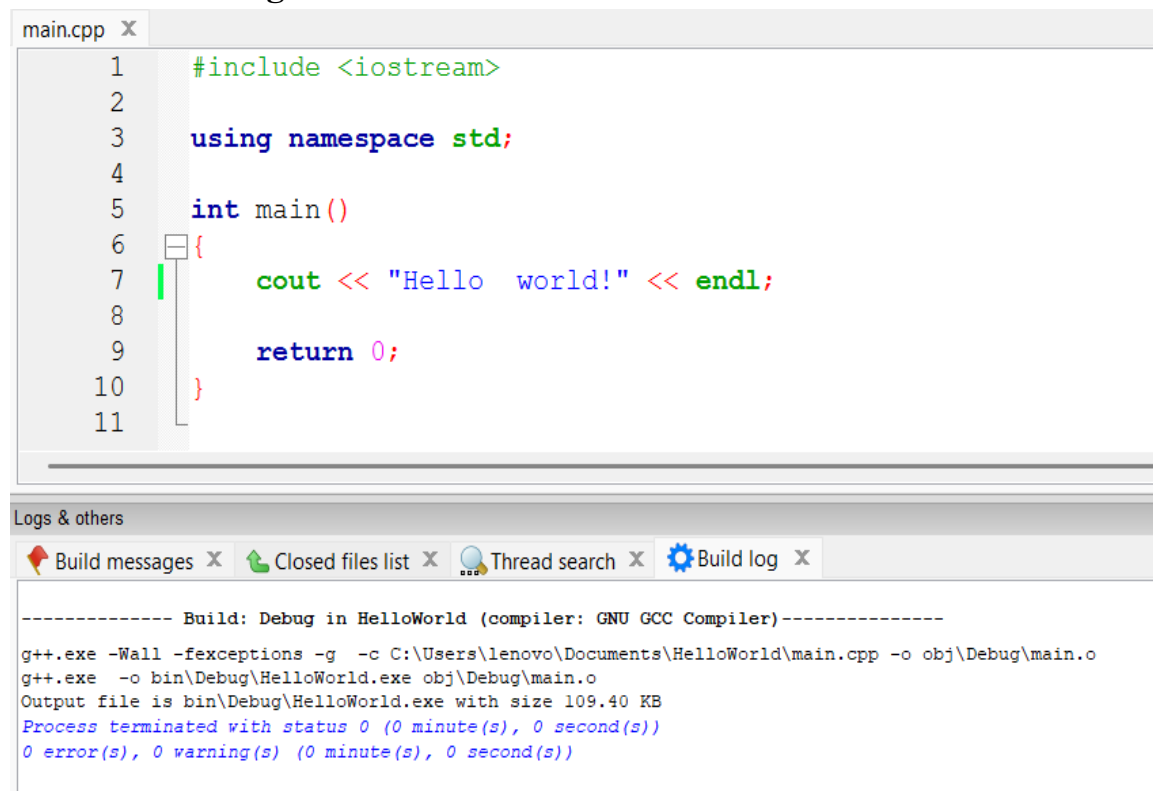
Part-1

Laboratory 2 – Programming Basics in C++

Lab Objectives

- Practice console input/output (cin, cout).
- Work with variables and arithmetic operations (Algorithm, Flowchart, Pseudocode).
- Improve writing readable code using comments and identifiers.
- If statement

1. Understanding Build



The screenshot displays a C++ IDE interface. The top pane shows the source code for 'main.cpp', which includes the `<iostream>` header, uses the `std` namespace, and contains a `main()` function that prints "Hello world!" and returns 0. The bottom pane, titled 'Logs & others', shows the build output for the 'HelloWorld' project using the GNU GCC Compiler. The output indicates a successful build with no errors or warnings.

```
main.cpp x
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      cout << "Hello world!" << endl;
8
9      return 0;
10 }
11

Logs & others
Build messages x Closed files list x Thread search x Build log x

----- Build: Debug in HelloWorld (compiler: GNU GCC Compiler)-----
g++.exe -Wall -fexceptions -g -c C:\Users\lenovo\Documents\HelloWorld\main.cpp -o obj\Debug\main.o
g++.exe -o bin\Debug\HelloWorld.exe obj\Debug\main.o
Output file is bin\Debug\HelloWorld.exe with size 109.40 KB
Process terminated with status 0 (0 minute(s), 0 second(s))
0 error(s), 0 warning(s) (0 minute(s), 0 second(s))
```

1.1 Compilation Step

```
g++.exe -Wall -fexceptions -g -c main.cpp -o main.o
```

CodeBlocks is using the **g++ compiler** to translate your code into machine code.

1. **-c** means compile only
2. **main.cpp** is your program
3. **main.o** is the compiled object file

1.2. Linking Step

```
g++.exe -o HelloWorld.exe main.o
```

The compiler now links everything into the final program:

1. **HelloWorld.exe** ← your final app
2. Located in `bin\Debug\`

1.3. Success Message

```
Output file is HelloWorld.exe
```

```
Process terminated with status 0
```

```
0 error(s), 0 warning(s)
```

- i. **Status 0** means everything is fine
- ii. No errors
- iii. No warnings

Your program built successfully.

1.4. Q: But where is the Output?

See `CMD` prompt screen.

2. Exercise 1: Print a Message

Write a program that prints the message: `Welcome to C++ Lab 2!`

```
#include <iostream>
using namespace std;

int main() {
    cout << "Welcome to C++ Lab 2!";
    return 0;
}
```

4. Exercise 2: Read Name & Address

Write a program that asks for the user's name and age, then prints the result.

```
#include <iostream>
using namespace std;

int main() {
    string name;

    // note the postal code here is integer type not string
    int address;
    cout << "Enter your name: ";
    cin >> name;
    cout << "Enter your address: ";
    cin >> address;

    cout << "Hello " << name << ", you are living at" << address << ".";
    return 0;
}
```

What can we change from the above program and keep same output as it is?

5. Exercise 3: Identifiers and Comments

Variables & Data Types
Int, float, char, string, bool.

```
int age = 20;  
float gpa = 3.7;  
char grade = 'A';  
string name = "Ali";  
bool isAStudent = true;
```

Rewrite Exercise 3 but follow these rules:

- Use meaningful identifiers (e.g., number1, number2, totalSum). Use camelCase for standardization
- Add comments explaining each part of your program.

Rules for Valid Identifiers

1. Must start with a letter (A–Z / a–z) or underscore `_`.
2. Can contain letters, digits (0–9), and underscores.
3. **Cannot** contain spaces or special characters like `@`, `#`, `$`, `%`, `-`.
4. **Cannot** be a C++ keyword (e.g., `int`, `while`, `return`).

Valid Identifiers (Correct)

1. totalScore
2. num1
3. _studentName

Invalid Identifiers (Incorrect)

1. 2value → starts with a digit
2. first-name → contains a hyphen
3. my score → contains a space

6. Exercise 4: Arithmetic Operations

Write algorithm and a C++ program that takes two numbers and prints their sum, difference, and product. Additionally give simple flowchart

6.1. flowchart

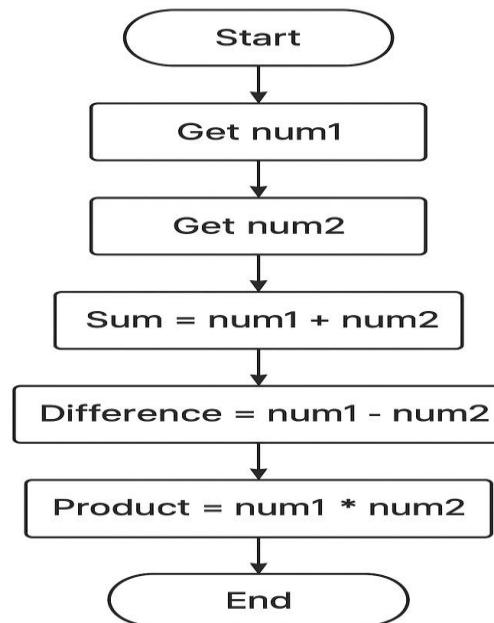


Fig.1 Simple Flowchart

6.2. Algorithm:

- Step 1: Start the program.
- Step 2: Declare two integer variables: num1 and num2.
- Step 3: Display the message: "Enter first number:"
- Step 4: Read the value and store it in num1.
- Step 5: Display the message: "Enter second number:"
- Step 6: Read the value and store it in num2.
- Step 7: Calculate the sum $\rightarrow \text{num1} + \text{num2}$.
- Step 8: Display: "Sum = " followed by the result.
- Step 9: Calculate the difference $\rightarrow \text{num1} - \text{num2}$.
- Step 10: Display: "Difference = " followed by the result.
- Step 11: Calculate the product $\rightarrow \text{num1} * \text{num2}$.
- Step 12: Display: "Product = " followed by the result.
- Step 13: End the program.

6.3. Optional Pseudocode

BEGIN

```
DECLARE num1 AS INTEGER
DECLARE num2 AS INTEGER

DISPLAY "Enter first number: "
INPUT num1

DISPLAY "Enter second number: "
INPUT num2

SET sum = num1 + num2
DISPLAY "Sum = ", sum

SET difference = num1 - num2
DISPLAY "Difference = ", difference

SET product = num1 * num2
DISPLAY "Product = ", product

END
```

6.4. Program

```
#include <iostream>
using namespace std;

int main() {
    // Dynamic values
    int num1, num2;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;

    cout << "Sum = " << num1 + num2 << endl;
    cout << "Difference = " << num1 - num2 << endl;
    cout << "Product = " << num1 * num2 << endl;
    return 0;
}
```

```
// Todo Add division operation  
// Reproduce with static values  
}
```

7. Task

Write a program that reads three numbers and prints the smallest.

8. Questions for Review

1. What is the difference between cin and cout?
2. List three valid and invalid identifiers.

Part-2

IF Condition + Comparison Programs

```
if (condition) {  
    // code to execute if condition is true  
}
```

Example of a condition:

```
✓ A > B  
✓ x == 10  
✓ grade != 'A'  
✓ age >= 18
```

Example 1: Check if a number is positive

```
int x;  
cout << "Enter a number: ";  
cin >> x;  
  
if (x > 0) {  
    cout << "The number is positive.";  
}
```

Syntax for if statement

important Points

- Uses *parentheses* around the condition.
- Uses *curly braces* `{}` to group code blocks.

- Ends each statement with a *semicolon*;
- Don't use *elif* it is for Python.

```
if (condition) {  
    // code to execute  
}  
else if (condition) {  
    // more code  
}  
else {  
    // fallback code  
}
```

Example 2: Write a program that Compare Two Numbers

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int A, B;  
  
    cout << "Enter two numbers: ";  
    cin >> A >> B;  
  
    if (A > B)  
        cout << "A is greater than B";  
    else if (A < B)  
        cout << "B is greater than A";  
    else  
        cout << "Both numbers are equal";  
  
    return 0;  
}
```

Example 3: Write C++ Program to check weather given number is Negative or Positive.

```
int x = 10;
```



```
if (x > 0) {  
    cout << "Positive";  
}  
else if (x == 0) {  
    cout << "Zero";  
}  
else {  
    cout << "Negative";  
}
```

Example 4: Write a program that the greatest among three numbers

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int A, B, C;  
    cout << "Enter three numbers: ";  
    cin >> A >> B >> C;  
  
    if (A > B && A > C)  
        cout << "A is the greatest";  
    else if (B > A && B > C)  
        cout << "B is the greatest";  
    else if (C > A && C > B)  
        cout << "C is the greatest";  
    else  
        cout << "Some numbers are equal";  
  
    return 0;  
}
```

Assignment-1: Write a C++ program that find the smallest among three numbers. (اخر وقت للتسليم 2025-12-7)